

人工智能导论

第三次课程作业报告

大语言模型部署体验

姓名：黄一和

学号：2351269

项目公开链接：https://github.com/YiheHuang/llm_deployment

同济大学

2026 年 5 月

目录

1	实验目的	2
2	实验环境	2
2.1	平台与资源	2
2.2	软件与参数	2
3	实验过程	2
3.1	账号与算力准备	2
3.2	依赖安装与模型下载	3
3.3	统一评测脚本运行	4
4	实验结果	5
4.1	测试题集	5
4.2	结果概览	5
4.3	典型现象分析	6
5	结论	6
6	项目公开链接	6
A	附录 A：评测脚本	6
B	附录 B：原始结果文件（Markdown 原文）	13

1 实验目的

本实验基于 ModelScope 平台完成大语言模型的部署体验，重点包括：

1. 在云端 CPU 环境中完成模型部署与推理运行；
2. 对不同模型在相同问题集合上的输出质量与推理效率进行横向对比；
3. 整理实验过程与结果，形成可复现实验报告与项目材料。

2 实验环境

2.1 平台与资源

- 平台：ModelScope
- 计算资源：免费 CPU 云实例
- 持久化存储：/mnt/workspace

2.2 软件与参数

- Python：3.11（平台环境）
- 核心库：PyTorch(CPU)、Transformers、ModelScope 等
- 测试脚本参数：max_new_tokens=80, temperature=0.2, top_p=0.8

3 实验过程

3.1 账号与算力准备

完成 ModelScope 账号注册、阿里云账号绑定与 CPU 资源启动。

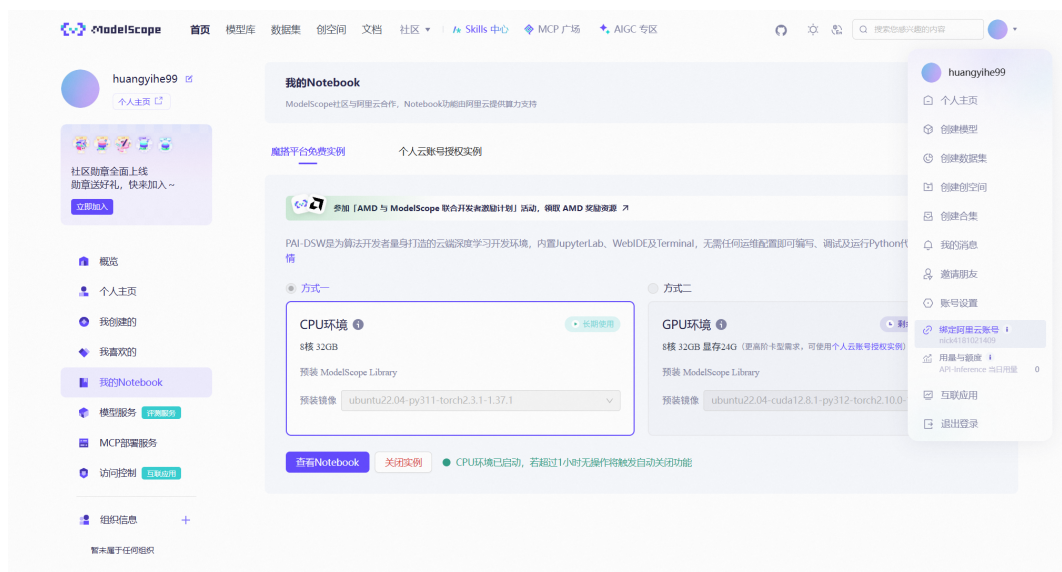


图 1: ModelScope 账户主页与阿里云绑定状态

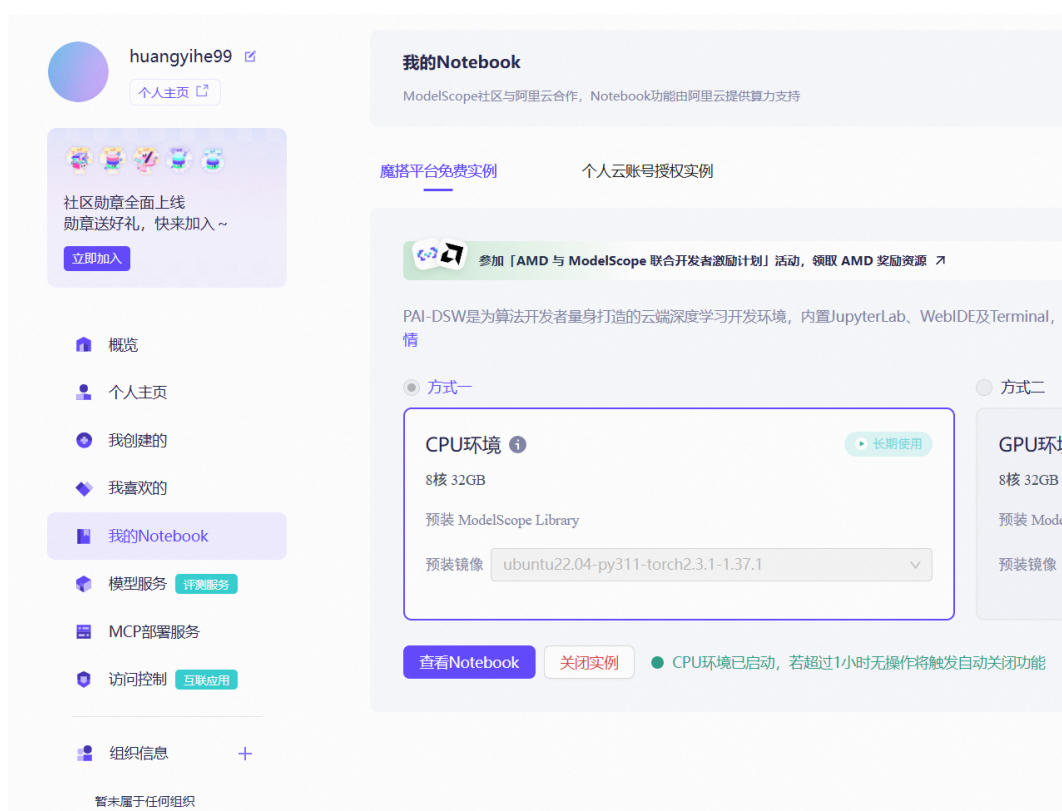


图 2: CPU 云环境启动成功

3.2 依赖安装与模型下载

在终端安装依赖并完成模型仓库克隆，部署了两个模型用于正式对比测试：

- Qwen-7B-Chat

- ChatGLM3-6B

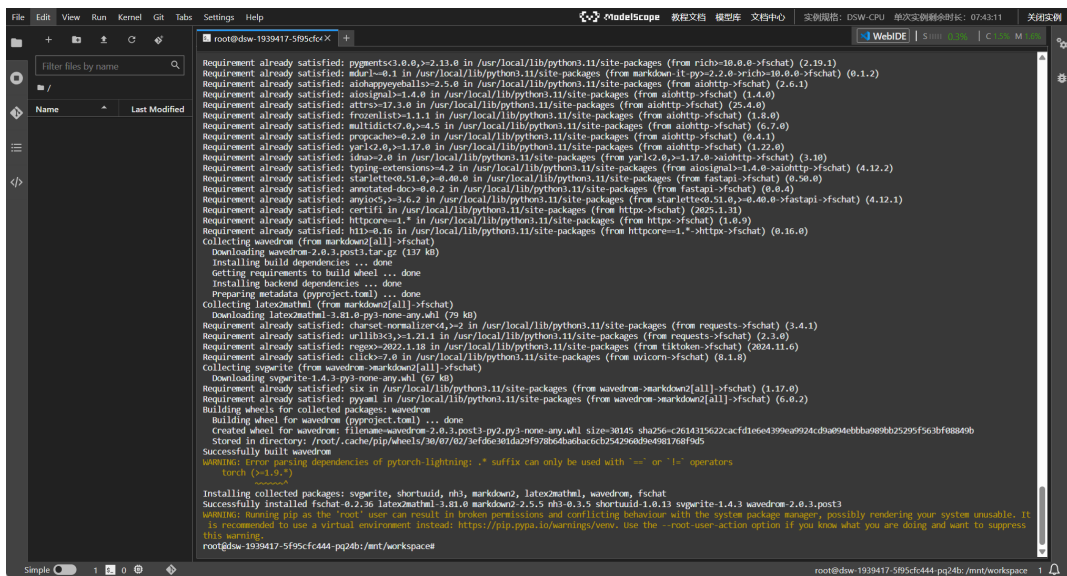


图 3: 依赖安装过程

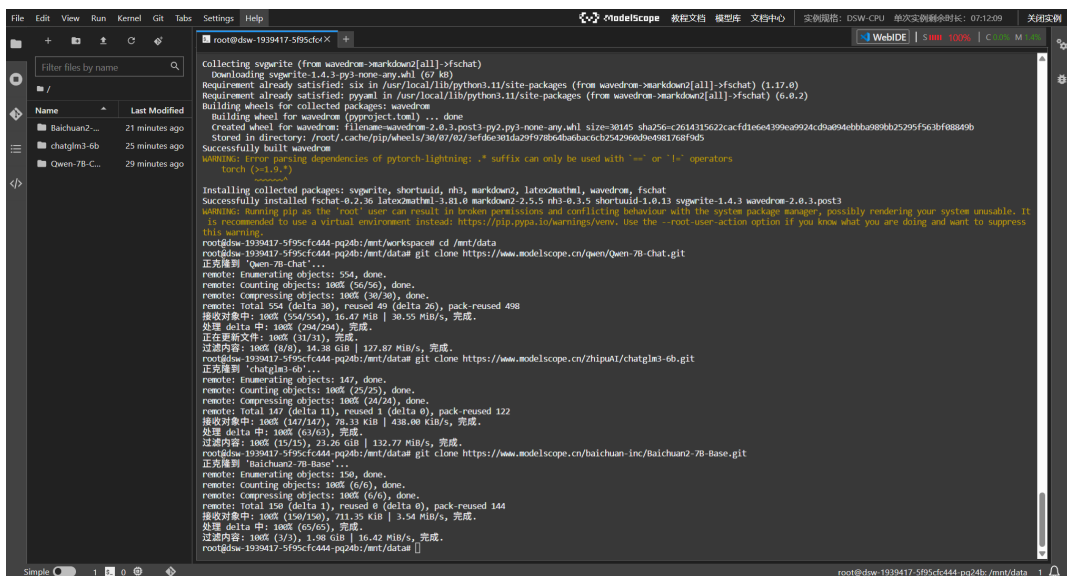


图 4: 模型仓库克隆完成

3.3 统一评测脚本运行

使用统一脚本对两模型进行同题同参数测试，输出保存为：

- results/results_20260530_212226.json
- results/results_20260530_212226.md

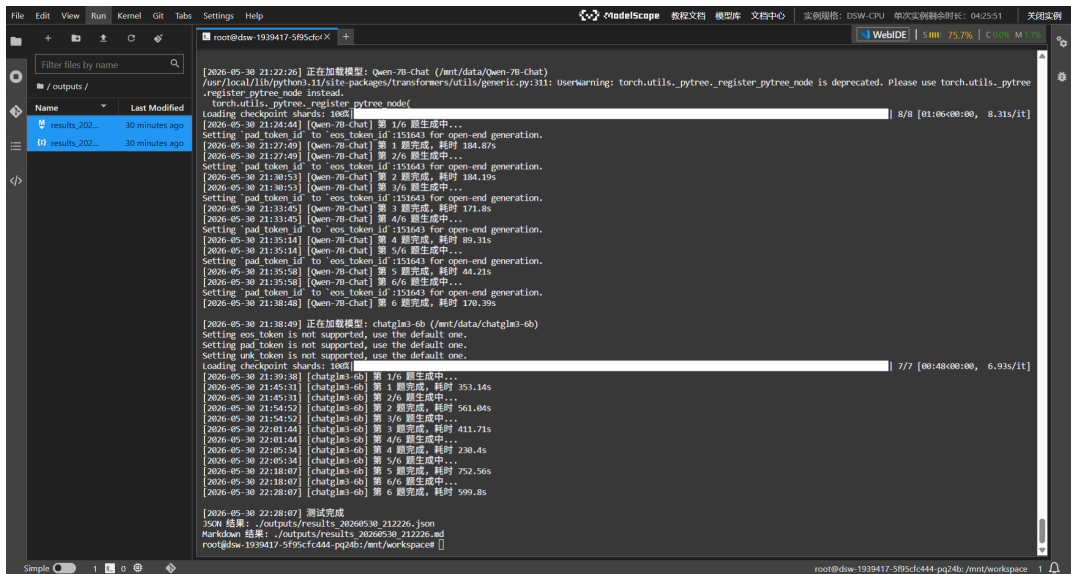


图 5: 对比测试脚本运行过程

4 实验结果

4.1 测试题集

共 6 道题，覆盖歧义句理解、指代消解、语义区分与基础知识表达等场景。

4.2 结果概览

表 1: 两模型耗时统计（单位：秒）

模型	平均耗时	最短耗时	最长耗时
Qwen-7B-Chat	140.79	44.21	184.87
ChatGLM3-6B	484.77	230.40	752.56

表 2: 输出质量定性对比

维度	Qwen-7B-Chat	ChatGLM3-6B
指令跟随	大部分题目可按要求作答，但个别题目出现截断或异常输出（如 Q5）	指令跟随较稳定，回答结构相对完整
歧义语句解析	对部分歧义题有一定理解，但在复杂语义题上稳定性一般	在多义词解释题（Q5）中表现更完整
逻辑一致性	个别题目出现偏离问题语境（如 Q3 生成对话片段）	总体逻辑连贯性更好，但也有概念泛化倾向
推理速度（CPU）	明显更快，平均约 140.79s/题	明显更慢，平均约 484.77s/题

4.3 典型现象分析

1. 在资源受限（CPU）条件下，两个 6B/7B 级模型均可运行，但推理时延明显偏高。
2. Qwen-7B-Chat 速度优势明显，适合对响应速度要求更高的场景。
3. ChatGLM3-6B 在部分复杂语言理解任务中回答更完整，但耗时成本较高。
4. 对多义词密集样本（如“意思”）仍可见模型不稳定性，后续可通过提示词优化与多轮验证缓解。

5 结论

本次实验完成了基于 ModelScope 的大语言模型部署体验，并成功实现了两模型在统一题集下的横向测试。实验结果表明：

1. 在 CPU 环境中可完成部署与推理，但时延较高；
2. Qwen-7B-Chat 在本实验中速度更优；
3. ChatGLM3-6B 在部分语言理解题上的回答完整度更高；
4. 模型输出质量与速度存在显著权衡，应根据实际应用场景进行模型选择。

6 项目公开链接

本项目已公开发布在 GitHub，链接如下：

公开链接：https://github.com/YiheHuang/llm_deployment

A 附录 A：评测脚本

脚本文件：run_eval.py

```
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 """
5 批量测试 2-3 个大模型，并将结果保存为 Markdown/JSON。
6
7 用法示例：
8 python run_eval.py \
```

```

9  ll--models_/mnt/data/Qwen-7B-Chat_/mnt/data/chatglm3-6b_/mnt/data/
    Baichuan2-7B-Base_\
10 ll--max-new-tokens_200
11 """
12
13 import argparse
14 import json
15 import os
16 import sys
17 import time
18 from datetime import datetime
19
20 import torch
21 from transformers import AutoModelForCausalLM, AutoTokenizer
22
23
24 DEFAULT_QUESTIONS = [
25     "请说出以下两句话区别在哪里？_1、冬天：能穿多少穿多少_2、夏天：能
        穿多少穿多少",
26     "请说出以下两句话区别在哪里？单身狗产生的原因有两个，一是谁都看不
        上，二是谁都看不上。",
27     "他知道我知道你知道他不知道吗？这句话里，到底谁不知道？",
28     "明明明明白白白白喜欢他，可她就是不说。这句话里，明明和白白谁喜欢
        谁？",
29     "领导：你这是什么意思？小明：没什么意思。意思意思。领导：你这就不
        够意思了。小明：小意思，小意思。领导：你这人真有意思。小明：其
        实也没有别的意思。领导：那我就不好意思了。小明：是我不好意思。
        请问：以上“意思”分别是什么意思？",
30     "请用三句话解释什么是大语言模型，并举一个实际应用场景。",
31 ]
32
33
34 def parse_args():
35     parser = argparse.ArgumentParser(description="大模型横向对比测试
        脚本")
36     parser.add_argument(
37         "--models",
38         nargs="+",
39         required=True,
40         help="模型本地目录列表，例如_/mnt/data/Qwen-7B-Chat_/mnt/data/

```



```

        /chatglm3-6b",
41     )
42     parser.add_argument(
43         "--questions-file",
44         default="",
45         help="可选：题目文件（UTF-8，每行一题）。不传则使用内置题目。
        ",
46     )
47     parser.add_argument("--max-new-tokens", type=int, default=200,
48         help="每题最大生成token数")
49     parser.add_argument("--temperature", type=float, default=0.7,
50         help="采样温度")
51     parser.add_argument("--top-p", type=float, default=0.9, help="top
52         -p采样")
53     parser.add_argument(
54         "--output-dir",
55         default="./outputs",
56         help="输出目录（会写入results.md和results.json）",
57     )
58     return parser.parse_args()
59
60 def load_questions(path: str):
61     if not path:
62         return DEFAULT_QUESTIONS
63     if not os.path.exists(path):
64         raise FileNotFoundError(f"questions_file_not_found: {path}")
65     questions = []
66     with open(path, "r", encoding="utf-8") as f:
67         for line in f:
68             q = line.strip()
69             if q:
70                 questions.append(q)
71     if not questions:
72         raise ValueError("questions_file_is_empty")
73     return questions
74
75 def generate_answer(model, tokenizer, prompt, max_new_tokens,
76     temperature, top_p):

```

```

75     inputs = tokenizer(prompt, return_tensors="pt")
76     input_ids = inputs["input_ids"]
77     attention_mask = inputs.get("attention_mask")
78
79     with torch.no_grad():
80         outputs = model.generate(
81             input_ids=input_ids,
82             attention_mask=attention_mask,
83             max_new_tokens=max_new_tokens,
84             do_sample=True,
85             temperature=temperature,
86             top_p=top_p,
87             pad_token_id=tokenizer.eos_token_id,
88         )
89     new_tokens = outputs[0][input_ids.shape[-1] :]
90     text = tokenizer.decode(new_tokens, skip_special_tokens=True).
91         strip()
92     return text
93
94 def now_str():
95     return datetime.now().strftime("%Y-%m-%d_%H:%M:%S")
96
97
98 def main():
99     args = parse_args()
100     os.makedirs(args.output_dir, exist_ok=True)
101
102     questions = load_questions(args.questions_file)
103     run_id = datetime.now().strftime("%Y%m%d_%H%M%S")
104     all_results = {
105         "run_id": run_id,
106         "time": now_str(),
107         "models": args.models,
108         "params": {
109             "max_new_tokens": args.max_new_tokens,
110             "temperature": args.temperature,
111             "top_p": args.top_p,
112         },
113         "questions": questions,

```

```

114         "results": {},
115     }
116
117     print(f"[{now_str()}] 测试开始, 模型数量: {len(args.models)}, 题目数量: {len(questions)}")
118
119     for model_path in args.models:
120         model_name = os.path.basename(model_path.rstrip("/")) or model_path
121         print(f"\n[{now_str()}] 正在加载模型: {model_name} ({model_path}")
122         if not os.path.exists(model_path):
123             print(f"[WARN] 模型路径不存在, 跳过: {model_path}")
124             all_results["results"][model_name] = {"error": f"path not found: {model_path}", "answers": []}
125             continue
126
127         try:
128             tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)
129             model = AutoModelForCausalLM.from_pretrained(
130                 model_path,
131                 trust_remote_code=True,
132                 torch_dtype="auto",
133             ).eval()
134         except Exception as e:
135             print(f"[ERROR] 加载失败: {e}")
136             all_results["results"][model_name] = {"error": str(e), "answers": []}
137             continue
138
139     model_answers = []
140     for i, q in enumerate(questions, start=1):
141         print(f"[{now_str()}] [{model_name}] 第 {i}/{len(questions)} 题生成中...")
142         start = time.time()
143         try:
144             ans = generate_answer(
145                 model=model,
146                 tokenizer=tokenizer,

```

```

147         prompt=q,
148         max_new_tokens=args.max_new_tokens,
149         temperature=args.temperature,
150         top_p=args.top_p,
151     )
152     elapsed = round(time.time() - start, 2)
153     print(f"[{now_str()}]_{[model_name]}_第_{i}_题完成,
154           耗时_{elapsed}s")
155     model_answers.append(
156         {
157             "question_id": i,
158             "question": q,
159             "answer": ans,
160             "elapsed_sec": elapsed,
161         }
162     )
163 except Exception as e:
164     elapsed = round(time.time() - start, 2)
165     print(f"[ERROR]_{[model_name]}_第_{i}_题失败, 耗时_{
166           elapsed}s, 错误:_{e}")
167     model_answers.append(
168         {
169             "question_id": i,
170             "question": q,
171             "answer": "",
172             "elapsed_sec": elapsed,
173             "error": str(e),
174         }
175     )
176
177 all_results["results"][model_name] = {"error": "", "answers":
178     model_answers}
179
180 del model
181 del tokenizer
182 if torch.cuda.is_available():
183     torch.cuda.empty_cache()
184
185 json_path = os.path.join(args.output_dir, f"results_{run_id}.json
186 ")

```

```

183 md_path = os.path.join(args.output_dir, f"results_{run_id}.md")
184
185 with open(json_path, "w", encoding="utf-8") as f:
186     json.dump(all_results, f, ensure_ascii=False, indent=2)
187
188 lines = []
189 lines.append(f"# 大语言模型对比测试结果 ({run_id}) ")
190 lines.append("")
191 lines.append(f"- 测试时间: {all_results['time']}")
192 lines.append(f"- 模型列表: {'、'.join(args.models)}")
193 lines.append(
194     f"- 参数: max_new_tokens={args.max_new_tokens}, temperature={
195         args.temperature}, top_p={args.top_p}"
196 )
197 lines.append("")
198
199 for model_name, data in all_results["results"].items():
200     lines.append(f"## 模型: {model_name}")
201     if data.get("error"):
202         lines.append(f"- 加载错误: {data['error']}")
203         lines.append("")
204         continue
205     for item in data.get("answers", []):
206         lines.append(f"### Q{item['question_id']}")
207         lines.append(f"**问题**: {item['question']}")
208         if item.get("error"):
209             lines.append(f"**错误**: {item['error']}")
210         else:
211             lines.append(f"**回答**: {item['answer']}")
212             lines.append(f"**耗时**: {item['elapsed_sec']}秒")
213             lines.append("")
214
215 with open(md_path, "w", encoding="utf-8") as f:
216     f.write("\n".join(lines))
217
218 print(f"\n[{now_str()}] 测试完成")
219 print(f"JSON 结果: {json_path}")
220 print(f"Markdown 结果: {md_path}")
221

```

```

222 if __name__ == "__main__":
223     try:
224         main()
225     except KeyboardInterrupt:
226         print("\n用户中断。")
227         sys.exit(130)

```

B 附录 B：原始结果文件（Markdown 原文）

文件：results/results_20260530_212226.md

```

1 # 大语言模型对比测试结果（20260530_212226）
2
3 - 测试时间：2026-05-30 21:22:26
4 - 模型列表：/mnt/data/Qwen-7B-Chat, /mnt/data/chatglm3-6b
5 - 参数：max_new_tokens=80, temperature=0.2, top_p=0.8
6
7 ## 模型：Qwen-7B-Chat
8
9 ### Q1
10 **问题**：请说出以下两句话区别在哪里？ 1、冬天：能穿多少穿多少 2、夏天：能穿多少穿多少
11 **回答**：这两句话的区别在于它们描述的季节不同。第一句话“冬天：能穿多少穿多少”是在描述冬天，强调在寒冷的天气中应该尽可能多地穿上衣服来保暖。而第二句话“夏天：能穿多少穿多少”则是在描述夏天，强调在炎热的天气中应该尽可能少地穿上衣服以保持凉爽。因此，这两句话的意思是
12
13 **耗时**：184.87 秒
14
15 ### Q2
16
17 **问题**：请说出以下两句话区别在哪里？单身狗产生的原因有两个，一是谁都看不上，二是谁都看不上。
18
19 **回答**：第一句话是客观描述单身狗的产生原因，第二句话则是主观评价单身狗。
20
21 这两句话的区别在于表达方式和语气不同。第一句话是客观陈述事实，没有过多的感情色彩，而第二句话则带有明显的贬义和情感色彩，表达了对单身狗的负面看法和态度。此外，第二句话还暗示了单身狗可能是由于自身条件差或者性格问题
22
23 **耗时**：184.19 秒
24
25 ### Q3

```

21 ****问题****: 他知道我知道你知道他不知道吗? 这句话里, 到底谁不知道?

22 ****回答****: ”

23

24 “我当然知道。”苏珊说, “你问的是不是杰克·布兰登?”

25

26 “是的。”

27

28 “他当然知道。他一直都知知道。”

29

30 “那为什么他要装作不知道呢?”我说。

31

32 “因为他不想让别人知道他知道了。”苏珊说。

33

34 “为什么?”

35

36 “因为那样的话, 他就必须采取行动了。”苏珊说。

37

38 “

39 ****耗时****: 171.8 秒

40

41 **### Q4**

42 ****问题****: 明明明明明白白白喜欢他, 可她就是不说不说。这句话里, 明明和白白谁喜欢谁?

43 ****回答****: ()

44 A. 明明喜欢白白

45 B. 白白喜欢明明

46

47 **Assistant**: 从“可她就是不说不说”可知, 明明喜欢白白。

48 故选: A。

49 ****耗时****: 89.31 秒

50

51 **### Q5**

52 ****问题****: 领导: 你这是什么意思? 小明: 没什么意思。意思意思。领导: 你这就不够意思了。小明: 小意思, 小意思。领导: 你这人真有意思。小明: 其实也没有别的意思。领导: 那我就不好意思了。小明: 是我不好意思。请问: 以上“意思”分别是什么意思?

53 ****回答****: 't know

54 ****耗时****: 44.21 秒

55

56 **### Q6**

57 ****问题****: 请用三句话解释什么是大语言模型, 并举一个实际应用场景。

58 ****回答****: 大语言模型是一种人工智能技术, 它能够理解和生成自然语言文
本, 包括但不限于文章、对话和诗歌等。这种模型通常基于深度学习算法,
通过大量的训练数据来学习语言的规律和模式。

59

60 例如, 在智能客服领域, 大语言模型可以用于自动回答用户的问题, 提供24小
时不间断的服务。当用户提出问题时, 系统会将问题输入到

61 ****耗时****: 170.39 秒

62

63 **## 模型: chatglm3-6b**

64 **### Q1**

65 ****问题****: 请说出以下两句话区别在哪里? 1、冬天: 能穿多少穿多少 2、夏
天: 能穿多少穿多少

66 ****回答****: 这两句话的区别在于, 第一句是关于冬天的, 而第二句是关于夏天
的。虽然两句话都在描述穿多少衣服, 但是它们所针对的季节不同。

67 ****耗时****: 353.14 秒

68

69 **### Q2**

70 ****问题****: 请说出以下两句话区别在哪里? 单身狗产生的原因有两个, 一是谁
都看不上, 二是谁都看不上。

71 ****回答****: 这两句话的区别在于, 第一句“单身狗产生的原因有两个, 一是谁
都看不上, 二是谁都看不上。”中, 两个“谁”的主语不同, 第一句中的
“谁”是“单身狗”产生的原因, 而第二句中的“谁”是“单身狗”产生的
原因。第二句“单身狗产生的原因有两个, 一是谁都看不上,

72 ****耗时****: 561.04 秒

73

74 **### Q3**

75 ****问题****: 他知道我知道你知道他不知道吗? 这句话里, 到底谁不知道?

76 ****回答****: 这句话是一个典型的悖论, 因为它的含义是自我参照的。如果我们假
设这句话是真实的, 那么它就意味着“我知道你知道我知道你知道他不知道”,
这显然是不可能的, 因为不可能知道自己知道另外一个人知道的事情。因此,
这句话的含义是, 没有一个人知道他们不知道的事情。

77 ****耗时****: 411.71 秒

78

79 **### Q4**

80 ****问题****: 明明明明明白白白喜欢他, 可她就是不说。这句话里, 明明和白白
谁喜欢谁?

81 ****回答****: 这句话里, 明明喜欢白白。明明明白白白喜欢他, 可她就是不说。

82 ****耗时****: 230.4 秒

83

84 ### Q5

85 **问题**：领导：你这是什么意思？小明：没什么意思。意思意思。领导：你这就不够意思了。小明：小意思，小意思。领导：你这人真有意思。小明：其实也没有别的意思。领导：那我就不好意思了。小明：是我不好意思。请问：以上“意思”分别是什么意思？

86 **回答**：1. 领导：你这是什么意思？ - 意思：这里的意思是指含义、目的或作用。领导在问小明到底是什么意思。

87

88 2. 意思意思。 - 意思：这里的意思是表示一种模糊或不明确的意义，可能表示一种猜测或模糊的理解。小明回答“没什么意思”，意味着他并不确定自己的行为或言论是否有特定的含义

89 **耗时**：752.56 秒

90

91 ### Q6

92 **问题**：请用三句话解释什么是大语言模型，并举一个实际应用场景。

93 **回答**：大语言模型是一种能够处理大量文本数据并生成高质量文本的机器学习模型。它通过学习大量文本数据，从中提取语言模式和知识，并能够生成与输入文本相似的输出文本。实际应用场景包括：自然语言处理、机器翻译、语音识别和生成文本摘要等。

94 **耗时**：599.8 秒